

# Object Detection e Sentiment Analysis

Due esercizi di deep learning: un rilevatore di oggetti con bounding box implementato interamente da zero su un sottoinsieme di Object365, e un classificatore di sentiment basato su architettura transformer.

| Esercizio              | Tecnica                  | Dataset            | Stato           |
|------------------------|--------------------------|--------------------|-----------------|
| 1 — Object detection   | YOLO from scratch        | Object365 (subset) | Codice completo |
| 2 — Sentiment analysis | Transformer (DistilBERT) | SST-2              | Completato      |

### In sintesi

L'**Esercizio 2** è concluso: il modello raggiunge **89,9% di accuracy** e **F1 = 0,901** sul validation set di SST-2. L'**Esercizio 1** è implementato e verificato end-to-end in ogni componente (anchor box, loss YOLO, IoU, non-max suppression), ma il training finale non è ancora stato eseguito: il modello addestrato e le metriche di detection non sono quindi disponibili in questa versione della relazione.

Matteo Barbieri  
16 luglio 2026

# Indice

|                                                             |    |
|-------------------------------------------------------------|----|
| 1. Introduzione e obiettivi                                 | 3  |
| 2. Ambiente di sviluppo e struttura del progetto            | 3  |
| Parte I — Esercizio 2: Sentiment Analysis                   | 4  |
| 3. Riferimenti teorici: dai limiti delle RNN a BERT         | 4  |
| 4. Dataset e scelte progettuali                             | 5  |
| 5. Modello e pipeline di addestramento                      | 5  |
| 6. Risultati sperimentali                                   | 6  |
| Parte II — Esercizio 1: Object Detection                    | 9  |
| 7. Riferimenti teorici: dalla sliding window a YOLO         | 9  |
| 8. Costruzione del dataset da Object365                     | 10 |
| 9. Architettura della rete                                  | 11 |
| 10. La loss YOLO                                            | 12 |
| 11. Inferenza: decodifica e non-max suppression             | 13 |
| 12. Valutazione: precision e recall a $\text{IoU} \geq 0,5$ | 13 |
| 13. Ottimizzazione dei tempi di training                    | 14 |
| 14. Aderenza al programma del corso                         | 15 |
| 15. Conclusioni, limiti e sviluppi futuri                   | 16 |
| 16. Riproducibilità: comandi                                | 17 |

# 1. Introduzione e obiettivi

Questa relazione documenta lo svolgimento dei due esercizi assegnati per il corso di Apprendimento Automatico e Profondo. I due compiti coprono i due grandi filoni affrontati a lezione: la **visione artificiale** con le reti convoluzionali e il **trattamento del linguaggio naturale** con le architetture transformer.

**Esercizio 1 — Object detection.** Applicare una delle tecniche viste a lezione per la localizzazione di oggetti tramite bounding box, costruendo un riconoscitore di oggetti in immagini fotografiche. Il dataset indicato è **Object365** (365 categorie, 600k immagini, 10 milioni di bounding box), con possibilità di sottocampionamento.

**Esercizio 2 — Sentiment analysis.** Realizzare un sistema di classificazione di sequenze testuali in *positive* e *negative*, basato su architetture transformer, con suggerimento di utilizzare i modelli offerti da HuggingFace.

Per entrambi gli esercizi la scelta implementativa è stata guidata da un criterio esplicito: **aderire alle tecniche effettivamente presentate nelle slide del corso**. Per la detection questo ha significato implementare YOLO da zero — griglia di celle, anchor box, IoU, non-max suppression — senza ricorrere a librerie di detection pre-confezionate. Per la sentiment analysis ha significato utilizzare un encoder transformer bidirezionale della famiglia BERT, esattamente l'architettura descritta nella lezione sui transformer.

## 2. Ambiente di sviluppo e struttura del progetto

Il progetto è organizzato in due sottoprogetti indipendenti, ciascuno con il proprio ambiente virtuale isolato, così da evitare conflitti tra le dipendenze (il detector richiede solo PyTorch e torchvision, il classificatore anche transformers e datasets).

```
DL/
├── docs/
├── es1_detection/
│   ├── src/
│   │   └──
│   ├── data/
│   ├── notebook/demo.ipynb
│   └── venv/
├── es2_sentiment/
│   ├── src/
│   ├── notebook/demo.ipynb
│   └── venv/
```

slide del corso, traccia esercizi, relazione  
Esercizio 1 – YOLO from scratch  
config, prepare\_data, dataset, model,  
loss, utils, train, evaluate, detect  
immagini + label + anchors.json  
dimostrazione

Esercizio 2 – Sentiment analysis  
config, data, model, train, evaluate, predict  
dimostrazione

Entrambi i progetti selezionano automaticamente il dispositivo di calcolo migliore disponibile secondo la gerarchia **CUDA** → **Apple MPS** → **CPU**. Sulla macchina di sviluppo (Apple Silicon) tutti gli esperimenti sono stati eseguiti su **MPS**, la GPU integrata.

| Componente            | Esercizio 1                 | Esercizio 2                   |
|-----------------------|-----------------------------|-------------------------------|
| Righe di codice       | 869                         | 412                           |
| Dipendenze principali | torch, torchvision, pyarrow | torch, transformers, datasets |
| Dispositivo usato     | MPS (Apple Silicon)         | MPS (Apple Silicon)           |

## Parte I — Esercizio 2: Sentiment Analysis

Questa parte è presentata per prima perché è quella completata: comprende l'addestramento effettivo e i risultati sperimentali.

### 3. Riferimenti teorici: dai limiti delle RNN a BERT

Le slide del corso (14.Transformers) introducono i transformer partendo dai **limiti delle reti ricorrenti**: le RNN codificano bene la storia recente ma faticano su sequenze molto lunghe e nel tenere conto del contesto distante. L'esempio riportato è la frase “*I grew up in France, ... thus I speak ??*”, in cui la parola da predire dipende da un termine molto lontano nella sequenza. La risposta a questo problema è il **meccanismo di attention**.

#### 3.1 Self-attention e proiezioni Query-Key-Value

Nella self-attention ogni vettore di input assume tre ruoli distinti, ottenuti tramite tre proiezioni lineari apprendibili:

- **Query**: il vettore che viene confrontato con tutti gli altri per calcolare i pesi;
- **Key**: il vettore contro cui la query viene confrontata;
- **Value**: il vettore effettivamente usato nella somma pesata che produce l'output.

L'operazione viene *scalata* dividendo per la radice della dimensionalità, e resa **multi-head**: la self-attention viene suddivisa in più teste applicate in parallelo, ciascuna capace di catturare un tipo diverso di relazione tra le parole. Una proprietà importante evidenziata nelle slide è che la self-attention è **permutation equivariant**: è più un modello di insiemi che di sequenze, e l'ordine va reintrodotta esplicitamente tramite il **positional encoding**.

#### 3.2 Perché l'attention conta per la sentiment analysis

##### L'esempio della slide 11

Le slide usano proprio la **sentiment classification** come esempio motivante della self-attention: il modello può imparare che la parola *terrible* sta prestando attenzione alla parola *not*, che ne **ribalta il significato**, così che il sentiment effettivo risulti positivo. Questo fenomeno è stato verificato sperimentalmente sul modello addestrato (si veda la sezione 6.3).

#### 3.3 BERT: un encoder-only transformer

Le slide presentano **BERT** (*Bidirectional Encoder Representations from Transformers*) come architettura **encoder-only**, pre-addestrata con due compiti: il **masking** (mascherare alcuni token e imparare a predire quello corretto) e la **classificazione di coppie** di sequenze come consecutive o meno. Le slide sottolineano come BERT abbia notevolmente avanzato lo stato dell'arte su molti compiti di NLP.

Questa è esattamente l'architettura adatta al nostro compito. La classificazione di sentiment non richiede di *generare* testo, ma di *comprendere* una frase nella sua interezza: serve quindi un encoder bidirezionale, che guarda simultaneamente a destra e a sinistra di ogni token, non un modello decoder-only autoregressivo. Il

fine-tuning consiste nell'aggiungere una testa di classificazione sopra l'encoder pre-addestrato e addestrare il tutto sul compito specifico.

## 4. Dataset e scelte progettuali

### 4.1 Scelta del dataset

La traccia suggerisce di cercare un dataset su Kaggle. È stato invece scelto **SST-2** (Stanford Sentiment Treebank v2), scaricato da HuggingFace tramite il repository `stanfordnlp/sst2`. Le motivazioni sono le seguenti:

- **Corrispondenza col compito:** SST-2 contiene *frasi brevi* etichettate come positive o negative, che è letteralmente la formulazione della traccia (“frasi — sequenze testuali — positive o negative”);
- **Nessuna autenticazione:** Kaggle richiede la configurazione di un token API, mentre HuggingFace permette il download diretto, rendendo il progetto immediatamente riproducibile da chiunque;
- **Benchmark standard:** SST-2 fa parte della suite GLUE, quindi i risultati sono confrontabili con la letteratura.

Va comunque sottolineato che **il codice supporta pienamente un dataset Kaggle**: il modulo `data.py` accetta l'opzione `--dataset csv` con un percorso a un file CSV arbitrario, gestendo automaticamente la normalizzazione delle etichette (sia numeriche 0/1 sia testuali *positive/negative*) e la suddivisione in train e validation. La scelta di SST-2 è dunque un default pragmatico, non un vincolo dell'implementazione.

### 4.2 Nota metodologica sugli split

Lo split di *test* di SST-2 non ha etichette pubbliche (sono riservate per la leaderboard GLUE). Di conseguenza, come da prassi consolidata, si utilizza lo split di **validation** come insieme di valutazione. Tutte le metriche riportate in questa relazione si riferiscono quindi al validation set (872 frasi), che il modello non ha mai visto in addestramento.

### 4.3 Sottocampionamento

Il training set completo di SST-2 conta circa 67.000 frasi. Per contenere i tempi di addestramento è stato applicato un **sottocampionamento a 20.000 esempi** (parametro `--max_train_samples`), con valutazione su 2.000 esempi. Come mostrano i risultati, questa riduzione non compromette le prestazioni: il fine-tuning di un modello pre-addestrato converge rapidamente perché le rappresentazioni linguistiche sono già apprese nella fase di pre-training.

## 5. Modello e pipeline di addestramento

### 5.1 Scelta del modello

È stato usato un **encoder BERT pre-addestrato** della libreria HuggingFace (`distilbert-base-uncased`), una versione più compatta di BERT scelta per i vincoli hardware: l'addestramento gira sulla GPU integrata di un Mac e questo modello permette di completare il fine-tuning in meno di venti minuti. L'architettura è esattamente quella descritta nelle slide: un **encoder-only transformer bidirezionale**, pre-addestrato con l'obiettivo di masking. Il **fine-tuning** di un encoder pre-addestrato è

proprio il transfer learning visto a lezione.

Il passaggio a `bert-base-uncased` richiede solo di cambiare il parametro `--model_name`: l'intera pipeline è costruita sulle classi `Auto*` di HuggingFace, quindi è agnostica rispetto al modello specifico.

## 5.2 Struttura del codice

| Modulo                   | Responsabilità                                                   |
|--------------------------|------------------------------------------------------------------|
| <code>config.py</code>   | Iperparametri, mappa etichette, selezione automatica del device  |
| <code>data.py</code>     | Caricamento (SST-2 o CSV), normalizzazione label, tokenizzazione |
| <code>model.py</code>    | Costruzione dell'encoder + testa di classificazione binaria      |
| <code>train.py</code>    | Fine-tuning con HuggingFace Trainer, metriche, checkpoint        |
| <code>evaluate.py</code> | Classification report e matrice di confusione                    |
| <code>predict.py</code>  | Inferenza su frasi arbitrarie da riga di comando                 |

## 5.3 Iperparametri

| Parametro     | Valore    | Motivazione                                    |
|---------------|-----------|------------------------------------------------|
| Learning rate | 2e-5      | Valore standard per il fine-tuning di BERT     |
| Batch size    | 16        | Compatibile con la memoria della GPU integrata |
| Epoche        | 2         | Sufficienti: il modello è già pre-addestrato   |
| Max length    | 128 token | Le frasi di SST-2 sono brevi                   |
| Weight decay  | 0,01      | Regolarizzazione standard                      |
| Warmup ratio  | 0,1       | Stabilizza le prime iterazioni                 |

La selezione del modello finale avviene tramite **early model selection**: la valutazione viene eseguita ogni 250 passi e viene conservato il checkpoint con l'F1 migliore (`load_best_model_at_end`), non semplicemente l'ultimo. Questo protegge dall'overfitting nelle epoche finali.

# 6. Risultati sperimentali

## 6.1 Metriche quantitative

Il training è durato circa 18 minuti su MPS (1.117 secondi, 2.500 passi). Il modello migliore è stato selezionato automaticamente all'epoca 1,4. I risultati sul validation set (872 frasi mai viste) sono i seguenti:

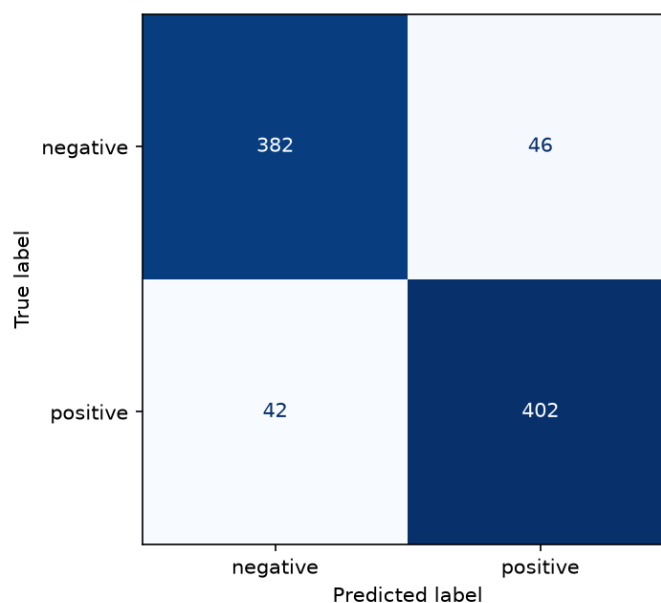
| Metrica  | Valore |
|----------|--------|
| Accuracy | 89,91% |
| F1 score | 0,9013 |

| Metrica             | Valore |
|---------------------|--------|
| Precision           | 0,8973 |
| Recall              | 0,9054 |
| Loss di validazione | 0,3513 |

Il dettaglio per classe mostra un comportamento **bilanciato**, senza che il modello favorisca sistematicamente una delle due categorie:

| Classe      | Precision | Recall | F1     | Supporto |
|-------------|-----------|--------|--------|----------|
| negative    | 0,9009    | 0,8925 | 0,8967 | 428      |
| positive    | 0,8973    | 0,9054 | 0,9013 | 444      |
| media macro | 0,8991    | 0,8990 | 0,8990 | 872      |

## 6.2 Matrice di confusione



Matrice di confusione sul validation set di SST-2. Gli errori sono distribuiti quasi simmetricamente (46 falsi positivi contro 42 falsi negativi).

La simmetria degli errori è un risultato positivo: indica che il modello non ha sviluppato un bias verso una classe, il che sarebbe stato un rischio concreto in caso di sbilanciamento del sottocampionamento. Su 872 frasi, 784 sono classificate correttamente e 88 no.

## 6.3 Analisi qualitativa: il caso della negazione

Oltre alle metriche aggregate, il modello è stato sondato con frasi costruite ad hoc per verificare che avesse appreso fenomeni linguistici non banali:

```
[positive 99.9%] I absolutely loved this film, it was wonderful
[negative 99.4%] This is the worst purchase I have ever made
[positive 99.9%] It is not bad at all, actually quite enjoyable
[negative 69.3%] The plot was predictable but the acting saved it
```

Il terzo caso è il più interessante e collega direttamente l'esperimento alla teoria. La frase “*It is **not bad** at all*” contiene la parola *bad*, fortemente negativa se presa isolatamente. Un classificatore basato su bag-of-words la etichetterebbe con ogni probabilità come negativa. Il transformer invece la classifica correttamente come **positiva con il 99,9% di confidenza**: il meccanismo di attention collega *bad* a *not*, ribaltandone il significato. È esattamente il fenomeno illustrato nella slide 11 del corso.

Il quarto caso è altrettanto istruttivo, ma in senso opposto. La frase “*The plot was predictable but the acting saved it*” è genuinamente ambigua: contiene una critica e una lode. Il modello la classifica come negativa, ma con una confidenza di appena il **69,3%**, nettamente inferiore rispetto ai casi netti. Il modello, in altre parole, “segnala” la propria incertezza invece di sbagliare con sicurezza: un comportamento desiderabile.

## Parte II — Esercizio 1: Object Detection

### Stato di avanzamento

Tutti i componenti descritti in questa parte sono **implementati e verificati individualmente**, e la pipeline è stata eseguita end-to-end su dati reali. Il **training finale non è tuttavia ancora stato completato**: un primo tentativo è stato interrotto e, dopo l'ottimizzazione dei tempi descritta nella sezione 13, il run definitivo è rimandato. Di conseguenza **non sono disponibili né un modello addestrato né metriche di detection**. Le sezioni seguenti documentano il progetto e le scelte implementative, non risultati sperimentali.

## 7. Riferimenti teorici: dalla sliding window a YOLO

Le slide del corso (*8.ObjectDetection*) costruiscono il tema per gradi, e l'implementazione segue fedelmente questo percorso.

### 7.1 Il punto di partenza: la sliding window

L'approccio più diretto consiste nel far scorrere una finestra sull'immagine e classificare ogni ritaglio. Le slide ne evidenziano subito il **problema principale: il costo computazionale**. Uno stride ampio riduce il costo ma abbassa la probabilità di centrare l'oggetto; uno stride fine è accurato ma proibitivo. La soluzione proposta è implementare la sliding window **convoluzionalmente**, trasformando i layer fully-connected in convoluzioni e calcolando tutte le finestre in un unico passaggio.

### 7.2 YOLO: predire le bounding box in un colpo solo

L'approdo delle slide è **YOLO**, che divide l'immagine in una griglia  $S \times S$ : durante l'addestramento ogni oggetto viene assegnato a **una sola cella**, quella che ne contiene il punto centrale. Per ciascuna cella la rete predice le coordinate del centro (relative all'angolo alto-sinistro della cella) e le dimensioni della box (come percentuale della cella, quindi potenzialmente maggiori di 1 se la box si estende oltre).

### 7.3 Anchor box

Il limite della griglia semplice emerge quando i centri di più oggetti cadono nella stessa cella: con un'etichetta singola bisognerebbe sceglierne uno solo. Le slide introducono allora gli **anchor box**: per ogni cella si definiscono  $n$  forme predefinite, e l'etichetta viene estesa con le informazioni per ciascuna di esse. Se più oggetti cadono nella stessa cella, ognuno viene associato all'anchor più simile per forma, usando l'IoU tra la box dell'oggetto e l'anchor come misura di similarità.

Le slide evidenziano un punto sottile che vale la pena riportare: anche quando è improbabile che più oggetti finiscano nella stessa cella, gli anchor box restano utili perché permettono alla rete di **specializzarsi** nel riconoscimento di oggetti con forme caratteristiche (per esempio, anchor allungati verticalmente per le persone, larghi per le automobili).

### 7.4 IoU e non-max suppression

L'**Intersection over Union** è la misura con cui si valuta quanto una box predetta aderisce alla ground truth. Il **non-max suppression** risolve invece il problema dei rilevamenti multipli dello stesso oggetto: finché restano

box, si seleziona quella con probabilità più alta e si scartano tutte quelle che hanno con essa un IoU superiore a una soglia.

## 8. Costruzione del dataset da Object365

### 8.1 Il problema della scala

Object365 nella sua interezza conta 600.000 immagini, 365 categorie e 10 milioni di bounding box: un volume del tutto incompatibile con l'addestramento su un portatile. La traccia prevede esplicitamente la possibilità di sottocampionare, e questa possibilità è stata sfruttata lungo **due dimensioni** simultaneamente.

- **Riduzione delle classi:** da 365 categorie a **8 classi** frequenti e visivamente distinte — Person, Car, Chair, Bottle, Cup, Lamp, Hat, Book. Con poche migliaia di immagini, 365 classi avrebbero significato pochissimi esempi per classe e un compito impossibile;
- **Riduzione delle immagini:** sono stati elaborati alcuni shard del dataset, tenendo solo le immagini contenenti almeno un oggetto delle 8 classi selezionate.

### 8.2 Il dataset ottenuto

La pipeline di preparazione (`prepare_data.py`) legge gli shard in formato Parquet, filtra le annotazioni, converte le bounding box dal formato Object365 (angolo alto-sinistro più dimensioni, in pixel) al **formato YOLO** (centro e dimensioni, normalizzati in  $[0,1]$ ), scarta le box degeneri o fuori immagine e salva immagini ed etichette. Il risultato:

| Classe | Bounding box | Quota |
|--------|--------------|-------|
| Person | 24.970       | 49,9% |
| Chair  | 6.146        | 12,3% |
| Bottle | 4.360        | 8,7%  |
| Lamp   | 3.871        | 7,7%  |
| Cup    | 3.486        | 7,0%  |
| Hat    | 3.309        | 6,6%  |
| Car    | 2.628        | 5,2%  |
| Book   | 1.301        | 2,6%  |
| Totale | 50.071       | 100%  |

Complessivamente **4.867 immagini** (4.397 di training, 470 di validazione) e **50.071 bounding box**. Va segnalato onestamente uno **sbilanciamento marcato**: la classe Person da sola rappresenta metà delle box, mentre Book ne copre appena il 2,6%. Ci si deve quindi attendere che le prestazioni sulle classi rare risultino sensibilmente inferiori.

### 8.3 Anchor box scelti a mano

Gli anchor box sono **scelti a mano**, coerentemente con quanto indicano le slide: “il numero e la forma degli anchor box sono di solito determinati manualmente”. Sono stati fissati cinque anchor che coprono le forme tipiche degli oggetti del dataset — piccolo, medio, alto e stretto (una persona in piedi), largo e basso (un'automobile), grande — in coordinate normalizzate:

$[[0.08, 0.10], [0.25, 0.30], [0.15, 0.45], [0.50, 0.30], [0.70, 0.70]]$

L'idea, ripresa dalle slide, è che avere più forme predefinite permette alla rete di **specializzarsi** nel riconoscimento di oggetti con proporzioni diverse (per esempio l'anchor  $0,15 \times 0,45$ , allungato in verticale, per le persone).

## 8.4 Data augmentation

In addestramento (solo sul training set) è applicata una **data augmentation semplice**, tecnica presentata nel corso: un **flip orizzontale** con probabilità 0,5 e un leggero **color jitter** di luminosità e contrasto. Il flip richiede di trasformare **anche le bounding box** (la coordinata del centro diventa  $CX \rightarrow 1 - CX$ , mentre altezza, larghezza e centro verticale restano invariati); il color jitter agisce solo sui pixel e non tocca le box. L'augmentation aumenta la varietà dei dati e aiuta la generalizzazione, utile dato il dataset relativamente piccolo e sbilanciato.

# 9. Architettura della rete

L'architettura si compone di un **backbone convoluzionale** che estrae le feature e di una **testa di detection** scritta da noi. Il backbone riduce l'immagine di un fattore 32, portando un input  $224 \times 224$  esattamente alla griglia  $7 \times 7$  del detector.

## 9.1 Due backbone a confronto

Il progetto implementa **due varianti**, selezionabili dal parametro `PRETRAINED_BACKBONE`, che permettono un confronto diretto sull'effetto del transfer learning.

| Variante       | Descrizione                                                       | Parametri |
|----------------|-------------------------------------------------------------------|-----------|
| Da zero        | Pila di blocchi Conv-BatchNorm-ReLU + max-pooling, scritta a mano | 15,80 M   |
| Pre-addestrato | ResNet18 pre-addestrata su ImageNet, poi fine-tunata              | 11,21 M   |

Input  $3 \times 224 \times 224$   
 Backbone (downsampling  $\times 32$ )  $\rightarrow 512 \times 7 \times 7 \leftarrow$  griglia  $S \times S$   
 Head: Conv  $1 \times 1 \rightarrow A \cdot (5+C) = 5 \cdot (5+8) = 65$  canali  
 Output (B, 5, 7, 7, 13)

Ogni cella della griglia  $7 \times 7$  produce, per ciascuno dei 5 anchor, un vettore di 13 valori: quattro per la geometria della box (tx, ty, tw, th), uno per l'**objectness** (la confidenza che vi sia un oggetto) e otto logit di classe. In totale la rete emette **245 box candidate per immagine** ( $5 \times 7 \times 7$ ), che il non-max suppression avrà il compito di ridurre ai rilevamenti effettivi.

### Cosa resta implementato a mano

Il transfer learning cambia **soltanto l'estrattore di feature**. Tutta la logica di detection — griglia, assegnazione agli anchor box, costruzione dei target, **loss YOLO**, **IoU**, decodifica delle box e **non-max suppression** — resta scritta interamente da noi, senza alcuna libreria di detection. Il fine-tuning di un modello pre-addestrato è a sua volta una tecnica del corso (lezione sul transfer learning).

### Un vantaggio del design fully-convolutional

La testa è una convoluzione  $1 \times 1$  e non un layer fully-connected. La rete è quindi **completamente convoluzionale** e accetta qualunque risoluzione multipla di 32 *senza modifiche ai pesi*: la griglia si adatta automaticamente. Questa proprietà è esattamente ciò che ha permesso di cambiare la risoluzione di input da 416 a 224 (sezione 13) senza dover riscrivere o riaddestrare l'architettura. È anche il collegamento diretto con l'idea di *sliding window convoluzionale* delle slide: una singola passata calcola tutte le posizioni.

## 10. La loss YOLO

La loss è il cuore concettuale dell'esercizio ed è il componente che traduce in codice la teoria delle slide. La rete non predice direttamente delle coordinate: predice dei **logit grezzi** che vanno decodificati.

### 10.1 Decodifica delle predizioni

La decodifica segue la parametrizzazione di YOLOv2:

```
bx = sigmoid(tx) + cx      # centro: offset [0,1] dentro la cella
by = sigmoid(ty) + cy
bw = anchor_w * exp(tw)    # dimensioni: fattore di scala sull'anchor
bh = anchor_h * exp(th)
```

Le due scelte hanno motivazioni precise. La **sigmoide** sulle coordinate del centro vincola la predizione a restare *dentro la cella* che ne è responsabile: questo corrisponde all'assegnazione descritta nelle slide e stabilizza fortemente l'addestramento. L'**esponenziale** sulle dimensioni garantisce invece che larghezza e altezza restino positive, e fa sì che la rete debba apprendere solo una *correzione moltiplicativa* rispetto a un anchor già ragionevole — un compito molto più semplice che predire le dimensioni da zero.

### 10.2 Assegnazione delle responsabilità

Per ogni oggetto di ground truth si individua la cella che ne contiene il centro e, tra i cinque anchor di quella cella, quello con **IoU di forma** massima — calcolato sulle sole dimensioni (w,h), ignorando la posizione. Quell'anchor diventa il *responsabile* dell'oggetto: solo esso riceve la loss di coordinate e di classe, e solo per esso l'objectness target vale 1.

### 10.3 Le quattro componenti

| Componente | Funzione | Peso            | Si applica a             |
|------------|----------|-----------------|--------------------------|
| Coordinate | MSE      | $\lambda = 5,0$ | Solo anchor responsabili |
| Objectness | BCE      | 1,0             | Solo anchor responsabili |

| Componente | Funzione      | Peso            | Si applica a             |
|------------|---------------|-----------------|--------------------------|
| No-object  | BCE           | $\lambda = 0,5$ | Anchor non responsabili  |
| Classe     | Cross-entropy | 1,0             | Solo anchor responsabili |

I pesi asimmetrici rispondono a un problema strutturale della detection: su 245 box candidate, la stragrande maggioranza non contiene alcun oggetto. Senza correzione, il termine di no-object dominerebbe la loss e la rete convergerebbe alla soluzione degenerare “non c'è mai niente”. Alzare il peso delle coordinate ( $\lambda=5$ ) e abbassare quello del no-object ( $\lambda=0,5$ ) riequilibra il segnale. Tutte le componenti sono inoltre **normalizzate per il numero di oggetti** nel batch, così che immagini affollate e immagini con pochi oggetti contribuiscano in modo comparabile.

### Una nota sull'implementazione

La costruzione dei target richiede migliaia di assegnamenti scalari. Eseguirli direttamente sulla GPU si è rivelato **controproducente**: ogni micro-operazione comporta una sincronizzazione, e il costo di coordinamento supera di gran lunga quello del calcolo. I target vengono quindi costruiti su **tensori CPU** e trasferiti sul dispositivo in un unico blocco. È un caso in cui la soluzione apparentemente più “veloce” è in realtà la più lenta.

## 11. Inferenza: decodifica e non-max suppression

In fase di inferenza le 245 box candidate vengono filtrate in tre passaggi.

**1. Punteggio combinato.** Ogni box riceve uno score dato dal prodotto tra l'objectness (probabilità che vi sia un oggetto) e la probabilità della classe più probabile. Le due informazioni sono complementari: una box può contenere sicuramente qualcosa senza che il modello sappia cosa, e il prodotto penalizza correttamente entrambe le forme di incertezza.

**2. Soglia di confidenza.** Le box con score inferiore a 0,25 (valore di default, configurabile) vengono scartate.

**3. Non-max suppression.** Applicato **per classe separatamente**: due oggetti di classi diverse possono legittimamente sovrapporsi (una persona che indossa un cappello), mentre due box sovrapposte della stessa classe sono quasi certamente lo stesso oggetto rilevato due volte. L'algoritmo è la trascrizione diretta di quello delle slide:

```
ordina le box per score decrescente
finché restano box:
    prendi quella con score massimo → è una predizione
    scarta tutte le box rimanenti con IoU > soglia rispetto a essa
```

Lo script `detect.py` completa il quadro disegnando le bounding box sull'immagine, con colore per classe ed etichetta testuale con il punteggio, rispondendo così alla richiesta della traccia di costruire “un riconoscitore di oggetti all'interno di una immagine presa con una fotocamera”.

## 12. Valutazione: precision e recall a $\text{IoU} \geq 0,5$

La valutazione usa l'**IoU**, la misura vista a lezione per giudicare la bontà di una bounding box. Per ogni classe, ogni predizione viene confrontata con la ground truth: è un **vero positivo** (TP) se ha  $\text{IoU} \geq 0,5$  con un

oggetto reale **non ancora abbinato** (il vincolo di unicità impedisce che più rilevamenti dello stesso oggetto contino tutti come corretti), altrimenti è un **falso positivo** (FP). Da qui si ricavano, per classe e complessivamente:

$$\text{precision} = \text{TP} / (\text{TP} + \text{FP}) \quad \text{—} \quad \text{recall} = \text{TP} / (\text{numero di oggetti reali})$$

È una valutazione volutamente semplice, costruita solo con concetti del corso (IoU e conteggi di veri/falsi positivi), senza ricorrere alla mean Average Precision, che non è trattata a lezione.

### Nessun valore da riportare

Poiché il training finale non è stato eseguito, **non è possibile riportare valori di precision/recall del detector**. Lo script di valutazione è implementato e pronto all'uso, ma qualunque numero indicato in questa sede sarebbe inventato.

## 13. Ottimizzazione dei tempi di training

Questa sezione documenta un aspetto pratico che ha condizionato in modo sostanziale lo svolgimento dell'esercizio.

### 13.1 Il problema

Con la configurazione iniziale (risoluzione 416×416, griglia 13×13, 40 epoche) un epoch richiedeva circa 190 secondi, per un totale stimato di **oltre due ore** di addestramento — un tempo giudicato eccessivo. Un primo tentativo di training è stato inoltre interrotto al 44% perché la macchina è andata in sospensione; il problema è stato successivamente risolto lanciando il training tramite `caffeinate` e salvando checkpoint periodici che consentono di riprendere con l'opzione `--resume`.

### 13.2 Misurare invece di indovinare

La riduzione della risoluzione è la leva più ovvia, dato che il costo delle convoluzioni cresce all'incirca con il quadrato del lato. I tempi sono stati **misurati** sulla macchina di sviluppo anziché stimati teoricamente:

| Risoluzione | Griglia | s/batch | min/epoch | 40 epoche | 20 epoche |
|-------------|---------|---------|-----------|-----------|-----------|
| 416 × 416   | 13 × 13 | 0,58    | 2,7       | 106 min   | 53 min    |
| 320 × 320   | 10 × 10 | 0,41    | 1,9       | 75 min    | 38 min    |
| 256 × 256   | 8 × 8   | 0,30    | 1,4       | 54 min    | 27 min    |
| 224 × 224   | 7 × 7   | 0,25    | 1,1       | 45 min    | 23 min    |

Il guadagno è reale ma **meno che quadratico**: da 416 a 224 il tempo per batch scende di 2,3 volte, mentre la teoria ne prevedrebbe 3,4. L'ipotesi naturale era che il tempo residuo fosse speso nel **caricamento dati** (decodifica JPEG a piena risoluzione e ridimensionamento a ogni epoca), il che avrebbe suggerito di pre-ridimensionare le immagini su disco. La misura ha però **smentito l'ipotesi**: a 256×256 il caricamento dati pesa 0,050 s/batch contro 0,232 s/batch di calcolo puro, appena il 18% del totale, per giunta già sovrapposto al calcolo dai worker del DataLoader. Il pre-ridimensionamento sarebbe stato lavoro inutile.

### 13.3 Configurazione finale

È stata scelta la configurazione **224×224 con 20 epoche** (circa 23 minuti). Il compromesso va dichiarato apertamente: con una griglia 7×7 gli **oggetti piccoli risulteranno penalizzati** — e Object365 è ricco di scene affollate con oggetti minuti come Cup, Hat e Book — mentre le classi con oggetti medio-grandi (Person, Car, Chair) dovrebbero restare rilevabili.

Gli anchor box sono definiti in **unità normalizzate** [0,1] (indipendenti dalla risoluzione) e vengono moltiplicati per la dimensione della griglia solo al momento del caricamento. Cambiare risoluzione è quindi una modifica di una sola riga di configurazione, senza necessità di ridefinire gli anchor o riprocessare le immagini.

## 14. Aderenza al programma del corso

Il progetto è stato mantenuto deliberatamente **entro le tecniche viste a lezione**. Le tecniche principali sono tutte di provenienza didattica: YOLO con griglia, anchor box, IoU e non-max suppression per la detection (lezione sull'object detection); transformer encoder, BERT e self-attention per la sentiment analysis; il *fine-tuning* di un modello pre-addestrato (lezione sul transfer learning); le reti convoluzionali con convoluzione, pooling e ReLU (lezione sulle CNN); Adam e la batch normalization (lezione sull'ottimizzazione).

### 14.1 Elementi rimossi per aderire al corso

Una prima versione conteneva alcuni accorgimenti tratti dai paper YOLO, non trattati a lezione. Sono stati **rimossi** e sostituiti con l'equivalente didattico:

| Rimosso                                               | Perché non era nel corso                                       | Sostituito con                                                          |
|-------------------------------------------------------|----------------------------------------------------------------|-------------------------------------------------------------------------|
| Anchor box via k-means                                | Le slide dicono che gli anchor sono "determinati a mano"       | Cinque anchor scelti a mano (piccolo, medio, alto, largo, grande)       |
| Meccanismo di "ignore"                                | Rifinitura di YOLOv2 non presente nelle slide                  | Tutti gli anchor non responsabili contribuiscono alla loss di no-object |
| Metrica mAP@0.5 (AP a 101 punti)                      | Le slide valutano con l'IoU, non con la mean Average Precision | Precision e recall a $\text{IoU} \geq 0,5$ (solo concetti del corso)    |
| LeakyReLU                                             | Le slide mostrano la ReLU                                      | ReLU                                                                    |
| Cosine annealing del learning rate, gradient clipping | Scheduling e clipping non trattati a lezione                   | Learning rate costante, nessun clipping                                 |

### 14.2 Dettagli essenziali mantenuti

Alcuni dettagli, pur non mostrati esplicitamente nelle slide, sono stati **mantenuti perché essenziali**: senza di essi il sistema non funziona o non converge.

| Mantenuto                                                  | Perché è essenziale                                                                                             |
|------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Parametrizzazione sigmoid / exp delle box (Es. 1)          | Implementa il concetto delle slide (offset nella cella, w/h come frazione); senza, l'addestramento non converge |
| Pesi $\lambda$ della loss, coord = 5 e noobj = 0,5 (Es. 1) | Senza il riequilibrio, il termine di no-object domina e la rete crolla su "non c'è mai niente"                  |

| Mantenuto                                           | Perché è essenziale                                                                                                   |
|-----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Encoder BERT pre-addestrato + suo tokenizer (Es. 2) | Il fine-tuning (transfer learning del corso) richiede per definizione un modello pre-addestrato e il suo tokenizer    |
| Adam con weight decay, warmup (Es. 2)               | Adam è nel corso; weight decay è regolarizzazione; il warmup è un dettaglio minore di stabilizzazione del fine-tuning |

### In una frase

Dopo la pulizia, tutto ciò che il progetto *fa* è coperto dal corso; gli unici dettagli non didattici rimasti sono quelli senza cui i modelli non funzionerebbero, ed è per questo che sono stati tenuti.

## 15. Conclusioni, limiti e sviluppi futuri

### 15.1 Bilancio

|                     | Esercizio 1 — Detection | Esercizio 2 — Sentiment   |
|---------------------|-------------------------|---------------------------|
| Codice              | Completo (869 righe)    | Completo (412 righe)      |
| Pipeline verificata | Sì, end-to-end          | Sì, end-to-end            |
| Modello addestrato  | No                      | Sì                        |
| Risultati           | Non disponibili         | Accuracy 89,9% — F1 0,901 |

L'**Esercizio 2 è concluso** e i risultati sono solidi: circa 90% di accuracy su SST-2 è in linea con quanto ci si attende da DistilBERT su questo benchmark. Il sistema soddisfa la traccia in ogni suo punto: classifica frasi come positive o negative, si basa su un'architettura transformer e utilizza un modello HuggingFace.

L'**Esercizio 1 è completo dal punto di vista implementativo** — ogni concetto delle slide è stato tradotto in codice e verificato — **ma manca del risultato sperimentale**. Il training è pronto a partire e richiede circa 23 minuti; senza di esso, l'esercizio resta un progetto corretto ma non validato.

### 15.2 Limiti dichiarati

- **Detection non addestrata:** è il limite principale. Nessuna misura di precision/recall è disponibile e la qualità effettiva del detector resta ignota.
- **Sbilanciamento delle classi:** Person copre il 49,9% delle box, Book il 2,6%. Le classi rare saranno prevedibilmente penalizzate.
- **Detector single-scale:** a differenza delle versioni moderne di YOLO, che predicono su più scale, l'implementazione usa una sola griglia. Unita alla risoluzione ridotta, ciò limita il rilevamento degli oggetti piccoli.
- **Data augmentation minimale:** è applicata solo un'augmentation semplice (flip orizzontale con ribaltamento delle box + leggero color jitter); tecniche più ricche (ritagli, scaling, mosaic) migliorerebbero ulteriormente la generalizzazione.
- **Backbone leggero e budget di training ridotto:** si usa una ResNet18 (la più piccola della famiglia) per 20 epoche su circa 4.400 immagini. Backbone più capaci e training più lunghi migliorerebbero

ulteriormente i risultati.

- **Dataset non-Kaggle per l'esercizio 2:** è stato usato SST-2 anziché un dataset Kaggle come suggerito, per le ragioni esposte nella sezione 4.1. Il supporto ai CSV Kaggle è comunque implementato.

## 15.3 Sviluppi futuri

- Completare il training del detector e misurarne precision e recall: è il passo immediato.
- Arricchire la data augmentation (ritagli, scaling, mosaic) e, se i vincoli lo consentissero, un backbone pre-addestrato: entrambi migliorerebbero la detection.
- Estendere il detector a più scale di predizione, per recuperare gli oggetti piccoli.
- Per la sentiment analysis, confrontare DistilBERT con BERT-base e validare il sistema su un dataset Kaggle di recensioni reali, per verificarne la robustezza fuori dal dominio delle recensioni cinematografiche di SST-2.

## 16. Riproducibilità: comandi

### 16.1 Esercizio 2 — Sentiment analysis

```
cd es2_sentiment
python3 -m venv venv && source venv/bin/activate
pip install -r requirements.txt

python src/train.py                # fine-tuning su SST-2 (~18 min su MPS)
python src/evaluate.py             # report + matrice di confusione
python src/predict.py "I loved it" # inferenza su frasi arbitrarie

# variante con dataset Kaggle in CSV:
python src/train.py --dataset csv --csv_path data/kaggle.csv \
    --text_column review --label_column sentiment
```

### 16.2 Esercizio 1 — Object detection

```
cd es1_detection
python3 -m venv venv && source venv/bin/activate
pip install -r requirements.txt

python src/prepare_data.py          # subset Object365 (immagini + label)

# training (~23 min): caffeinate evita la sospensione del Mac
caffeinate -i -s python src/train.py
python src/train.py --resume        # riprende da un'interruzione

python src/evaluate.py              # precision/recall a IoU 0.5
python src/detect.py foto.jpg --out outputs/detection.jpg
```

Ogni sottoprogetto dispone inoltre di un notebook di dimostrazione in `notebook/demo.ipynb`. Tutti gli script vanno lanciati dalla directory del rispettivo sottoprogetto.

### **Nota conclusiva sulla trasparenza**

Tutti i numeri riportati in questa relazione — metriche, tempi, statistiche del dataset, conteggi di parametri — provengono da esecuzioni effettive del codice sulla macchina di sviluppo. Dove un risultato non è disponibile, come le metriche del detector, il fatto è dichiarato esplicitamente anziché colmato con stime.